

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Thread & Concorrenza : Exam Exercises

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

TdE: 07/02/2022 Quesito **a**

Esercizio 2

Si consideri nuovamente un'implementazione di `JobsDB` che fa uso di un `Set` per memorizzare le richieste di lavoro e di una `List` per memorizzare le offerte di lavoro, come mostrato di seguito.

```
public class JobsDB {
  private Set<JobRequest> requests;
  private List<JobOffer> offers;
      }
```

Domanda a)

Si completi l'implementazione considerando solo i metodi `getRequests`, `getOffers`, `addRequest`, `addOffer` e garantendo thread-safety, ovvero facendo in modo che diversi thread possano invocare i metodi in parallelo senza creare problemi di concorrenza. **Si favoriscano soluzioni che massimizzano le possibilità di esecuzione parallela da parte dei thread che invocano i diversi metodi.**

TdE: 07/02/2022 Quesito **a**

- getRequests(): restituisce un HashSet di richieste.
- getOffers(): restituisce un array di offerte.
- addRequests(): aggiunge una richiesta all'hashset di richieste
- addOffers(): aggiunge una JobOffer all'array di offerte e restituisce il **set** di richieste che metchano quell'offerta

Potete assumere che esista un metodo **matches()** delle requests che restituisce TRUE se l'offerta è coerente con la richiesta

TdE: 07/02/2022 Quesito **a - soluzione**

```
public class JobsDB {
  ...
  public Set<JobRequest> getRequests() {
    synchronized(requests) { return new HashSet<>(requests); } }

  public List<JobOffer> getOffers() {
    synchronized(offers) { return new ArrayList<>(offers); }  }

  public Set<JobRequest> addOffer(JobOffer offer) {
    synchronized(offers) { offers.add(offer); }
    synchronized(requests) {
      return requests.stream()
                     .filter(req -> req.matches(offer))
                     .collect(Collectors.toSet());
    }  }  }
}
```

TdE: 07/02/2022 Quesito **a - soluzione**

Devo usare i blocchi **synchronized**, per garantire il più alto parallelismo possibile

```
public class JobsDB {
   ...
   public Set<JobRequest> getRequests() {
      synchronized(requests) { return new HashSet<>(requests); } }

   public List<JobOffer> getOffers() {
      synchronized(offers) { return new ArrayList<>(offers); }   }

   public Set<JobRequest> addOffer(JobOffer offer) {
      synchronized(offers) { offers.add(offer); }
      synchronized(requests) {
         return requests.stream()
                        .filter(req -> req.matches(offer))
                        .collect(Collectors.toSet());
      }   }   }
}
```

TdE: 07/02/2022 Quesito **a - soluzione**

Devo restituire una **copia** delle mie Collections,
Cosi che si possa accedere tranquillamente

```
public class JobsDB {
  ...
  public Set<JobRequest> getRequests() {
    synchronized(requests) { return new HashSet<>(requests); } }

  public List<JobOffer> getOffers() {
    synchronized(offers) { return new ArrayList<>(offers); }  }

  public Set<JobRequest> addOffer(JobOffer offer) {
    synchronized(offers) { offers.add(offer); }
    synchronized(requests) {
      return requests.stream()
                     .filter(req -> req.matches(offer))
                     .collect(Collectors.toSet());
    } } }
}
```

TdE: 07/02/2022 Quesito **a - soluzione**

Uso due blocchi **synchronized** diversi, uno per ogni risorsa a cui devo accedere

```
public class JobsDB {
  ...
  public Set<JobRequest> getRequests() {
    synchronized(requests) { return new HashSet<>(requests); } }

  public List<JobOffer> getOffers() {
    synchronized(offers) { return new ArrayList<>(offers); }  }

  public Set<JobRequest> addOffer(JobOffer offer) {
    synchronized(offers) { offers.add(offer); }
    synchronized(requests) {
      return requests.stream()
                     .filter(req -> req.matches(offer))
                     .collect(Collectors.toSet());
    } } }
}
```

TdE: 07/02/2022 Quesito **b.**

Domanda b)

Si implementi ora anche il metodo `getOffer` facendo in modo che, invece di ritornare `null`, sospenda il chiamante nel caso in cui nessuna offerta risulta compatibile con la richiesta. Risulta necessario modificare anche le implementazioni di altri metodi? Se si, come?

Potete assumere che esista un metodo **noneMatch()** che restituisce True se l'offerta **non** è coerente con nessuna richiesta

TdE: 07/02/2022 Quesito **b - soluzione**

```
public class JobsDB {
  ...
  public JobOffer getOffer(JobRequest req) {
    synchronized(offers) {
      while (offers.stream().noneMatch(offer -> req.matches(offer))) {
        offers.wait();
      }
      return offers.stream().findFirst(offer -> req.matches(offer)).get();
    }
  }
```

TdE: 07/02/2022 Quesito **b - soluzione**

```
public class JobsDB {
  ...
  public JobOffer getOffer(JobRequest req) {
    synchronized(offers) {
      while (offers.stream().noneMatch(offer -> req.matches(offer))) {
        offers.wait();
      }
      return offers.stream().findFirst(offer -> req.matches(offer)).get();
    }
  }
```

Ho aggiunto una wait()…quindi dovrò aggiungere una NotifyAll()
DOVE?

TdE: 07/02/2022 Quesito **b - soluzione**

```
public class JobsDB {
  ...
  public JobOffer getOffer(JobRequest req) {
    synchronized(offers) {
      while (offers.stream().noneMatch(offer -> req.matches(offer))) {
        offers.wait();
      }
      return offers.stream().findFirst(offer -> req.matches(offer)).get();
    }
  }
```

Ho aggiunto una wait()…quindi dovrò aggiungere una NotifyAll()
DOVE? **Nel metodo AddOffer, cosi che vengano ricontrollate tutte le offerte**

TdE: 07/02/2022 Quesito **b - soluzione**

```
public class JobsDB {
  ...
  public JobOffer getOffer(JobRequest req) {
    synchronized(offers) {
      while (offers.stream().noneMatch(offer -> req.matches(offer))) {
        offers.wait();
      }
      return offers.stream().findFirst(offer -> req.matches(offer)).get();
    }
  }
```

PRIMA

```
public Set<JobRequest> addOffer(JobOffer offer) {
  synchronized(offers) { offers.add(offer); }
  synchronized(requests) {
    return requests.stream()
                   .filter(req -> req.matches(offer))
                   .collect(Collectors.toSet());
  } } }
```

TdE: 07/02/2022 Quesito **b - soluzione**

```
public class JobsDB {
  ...
  public JobOffer getOffer(JobRequest req) {
    synchronized(offers) {
      while (offers.stream().noneMatch(offer -> req.matches(offer))) {
        offers.wait();
      }
      return offers.stream().findFirst(offer -> req.matches(offer)).get();
    }
  }
```

DOPO

```
public Set<JobRequest> addOffer(JobOffer offer) {
  synchronized(offers) {
    offers.add(offer);
    offers.notifyAll();
  }
  synchronized(requests) {
    return requests.stream()
                   .filter(req -> req.matches(offer))
                   .collect(Collectors.toSet());
  } } }
```

TdE 28/08/2023

Esercizio 2

Si consideri la seguente classe `SensorReadings` per memorizzare le letture di un sensore ambientale:

```
public class SensorReadings {
  private int id;
  private List<Double> data;
  private int daysInUse;
  public SensorReadings(int id) {
    this.id = id;
    data = new ArrayList<Double>();
    daysInUse = 0;
  }
  public void addReading(double d) {
    synchronized(data) { data.add(d); }
  }
  public double getReading(int index) {
    synchronized(data) { return data.get(index); }
  }
  public void changeAllReadings(List<Double> newData) {
    synchronized(data) { data = newData; }
  }
  public synchronized void newDayStarted() {
    daysInUse++;
  }
}
```

TdE 28/08/2023 – Quesito **a**

Esercizio 2

Si consideri la seguente classe `SensorReadings` per memorizzare le letture di un sensore ambientale:

```
public class SensorReadings {
  private int id;
  private List<Double> data;
  private int daysInUse;
  public SensorReadings(int id) {
    this.id = id;
    data = new ArrayList<Double>();
    daysInUse = 0;
  }
  public void addReading(double d) {
    synchronized(data) { data.add(d); }
  }
  public double getReading(int index) {
    synchronized(data) { return data.get(index); }
  }
  public void changeAllReadings(List<Double> newData) {
    synchronized(data) { data = newData; }
  }
  public synchronized void newDayStarted() {
    daysInUse++;
  }
}
```

Domanda a)
La classe è correttamente sincronizzata? In caso negativo, si spieghi quali siano i problemi e si proponga una soluzione, a tali problemi, che **massimizzi** il **parallelismo** tra i metodi

TdE 28/08/2023 – Quesito **a**

Esercizio 2

Si consideri la seguente classe `SensorReadings` per memorizzare le letture di un sensore ambientale:

```
public class SensorReadings {
  private int id;
  private List<Double> data;
  private int daysInUse;
  public SensorReadings(int id) {
    this.id = id;
    data = new ArrayList<Double>();
    daysInUse = 0;
  }
  public void addReading(double d) {
    synchronized(data) { data.add(d); }
  }
  public double getReading(int index) {
    synchronized(data) { return data.get(index); }
  }
  public void changeAllReadings(List<Double> newData) {
    synchronized(data) { data = newData; }
  }
  public synchronized void newDayStarted() {
    daysInUse++;
  }
}
```

Domanda a)
La classe è correttamente sincronizzata? In caso negativo, si spieghi quali siano i problemi e si proponga una soluzione, a tali problemi, che **massimizzi** il **parallelismo** tra i metodi

TdE 28/08/2023 – Quesito **a - soluzione**

Esercizio 2

Si consideri la seguente classe `SensorReadings` per memorizzare le letture di un sensore ambientale:

```
public class SensorReadings {
  private int id;
  private List<Double> data;
  private int daysInUse;
  public SensorReadings(int id) {
    this.id = id;
    data = new ArrayList<Double>();
    daysInUse = 0;
  }
  public void addReading(double d) {
    synchronized(data) { data.add(d); }
  }
  public double getReading(int index) {
    synchronized(data) { return data.get(index); }
  }
  public void changeAllReadings(List<Double> newData) {
    synchronized(data) { data = newData; }
  }
  public synchronized void newDayStarted() {
    daysInUse++;
  }
}
```

Il **problema** è che nel metodo «change all Reading» si sta cambiando l'oggetto su cui viene effettuata la sincronizzazione. **Soluzione** a tale problema, per massimizzare il parallelismo, è aggiungere un oggetto **dataLock** che **non** cambia mai.

TdE 28/08/2023 – Quesito **a - soluzione**

```
public class SensorReadings {
  private int id;
  private List<Double> data;
  private Object dataLock;
  private int daysInUse;
  public SensorReadings(int id) {
    this.id = id;
    data = new ArrayList<Double>();
    dataLock = new Object();
    daysInUse = 0;
  }
  public void addReading(double d) {
    synchronized(dataLock) { data.add(d); }
  }
  public double getReading(int index) {
    synchronized(dataLock) { return data.get(index); }
  }
  public void changeAllReadings(List<Double> newData) {
    synchronized(dataLock) { data = newData; }
  }
  public synchronized void newDayStarted() {
    daysInUse++;
  }
}
```

TdE 28/08/2023 – Quesito **b**

Domanda b)

Si aggiunga alla classe `SensorReadings` un nuovo metodo pubblico: `void waitfor30()` che sospende il chiamante se ci sono meno di 30 letture nella lista `data`.

TdE 28/08/2023 – Quesito **b**

Domanda b)

Si aggiunga alla classe `SensorReadings` un nuovo metodo pubblico: `void waitfor30()` che sospende il chiamante se ci sono meno di 30 letture nella lista `data`.

```
public void waitfor30() throws InterruptedException {
  synchronized(dataLock) {
    while(data.size()<30) dataLock.wait();
  }
}
```

TdE 28/08/2023 – Quesito **b soluzione**

Domanda b)

Si aggiunga alla classe `SensorReadings` un nuovo metodo pubblico: `void waitfor30()` che sospende il chiamante se ci sono meno di 30 letture nella lista `data`.

```
public void waitfor30() throws InterruptedException {
   synchronized(dataLock) {
     while(data.size()<30) dataLock.wait();
   }
}
```

Ho aggiunto una wait, quindi occorre aggiungere Una chiamata a **notifyAll**() alla fine dei metodi che aggiungono letture, ovvero: **addReadings**() e **changeAllReadings**()

TdE 28/08/2023 – Quesito **c**

Domanda c)

Si aggiunga un nuovo metodo pubblico: `void readyToBeReplaced()` che sospende il chiamante fino a quando non ci siano almeno 10000 letture nella lista `data` oppure siano passati almeno 365 giorni di utilizzo del sensore. Si chiarisca se e come deve cambiare la sincronizzazione degli altri metodi.

TdE 28/08/2023 – Quesito **c - soluzione**

Domanda c)

Si aggiunga un nuovo metodo pubblico: `void readyToBeReplaced()` che sospende il chiamante fino a quando non ci siano almeno 10000 letture nella lista `data` oppure siano passati almeno 365 giorni di utilizzo del sensore. Si chiarisca se e come deve cambiare la sincronizzazione degli altri metodi.

Soluzione:

La condizione di sospensione riguarda tanto l'attributo data quanto l'attributo daysInUse, il cui accesso è stato fin qui protetto usando due "lock" diversi (dataLock e this). Per come funziona la sincronizzazione di Java questo approccio non si può più seguire ed è quindi necessario usare un unico "lock" per tutto. Ad esempio, scegliendo this, si può giungere alla seguente soluzione:

TdE 28/08/2023 – Quesito **c - soluzione**

```
class SensorReadings {
  private int id;
  private List<Double> data;
  private int daysInUse;
  public SensorReadings(int id) {
    this.id = id;
    data = new ArrayList<Double>();
    daysInUse = 0;
  }
  public synchronized void addReading(double d) {
    data.add(d); notifyAll();
  }
  public synchronized double getReading(int index) {
    return data.get(index);
  }
  public synchronized void changeAllReadings(List<Double> newData) {
    data = newData; notifyAll();
  }
  public synchronized void newDayStarted() {
    daysInUse++; notifyAll();
  }
  public synchronized void waitfor30() throws InterruptedException {
    while(data.size()<30) wait();
  }
  public synchronized void readyToBeReplaced() throws InterruptedException {
    while(data.size()<10000 || daysInUse < 365) wait();
  }
}
```

TdE 07/06/2023

Si consideri la seguente classe **DataBuffer** che contiene dati numerici che possono essere scritti e letti a partire da una particolare posizione detta “cursore”.

```
public class DataBuffer {
  private double[] data;
  private int cursor;

  public DataBuffer() { data = new double[100]; cursor = 0; }

  public double getDataAtCursor() {
    synchronized(data) { return data[cursor]; }
  }

  public void setDataAtCursor(double d) {
    synchronized(data) { data[cursor] = d; }
  }

  public void setCursorPos(int pos) {
    synchronized(cursor) { cursor = pos; }
  }
}
```

TdE 07/06/2023 – Quesito **a.**

```
public class DataBuffer {
  private double[] data;
  private int cursor;

  public DataBuffer() { data = new double[100]; cursor = 0; }

  public double getDataAtCursor() {
    synchronized(data) { return data[cursor]; }
  }

  public void setDataAtCursor(double d) {
    synchronized(data) { data[cursor] = d; }
  }

  public void setCursorPos(int pos) {
    synchronized(cursor) { cursor = pos; }
  }
}
```

Quesito a.
Il codice `e correttamente sincronizzato? Motivare la propria la risposta e in caso di risposta negativa modificare un solo metodo della classe affinchè la sincronizzazione risulti corretta.

TdE 07/06/2023 – Quesito **a. - soluzione**

```
public class DataBuffer {
  private double[] data;
  private int cursor;

  public DataBuffer() { data = new double[100]; cursor = 0; }

  public double getDataAtCursor() {
    synchronized(data) { return data[cursor]; }
  }

  public void setDataAtCursor(double d) {
    synchronized(data) { data[cursor] = d; }
  }

  public void setCursorPos(int pos) {
    synchronized(cursor) { cursor = pos; }
  }
}
```

I metodi che accedono all'attributo cursor acquisiscono lock su oggetti diversi. Inoltre, il metodo setCursorPos acquisisce il lock su un oggetto di un tipo base (int), il che non `e consentito dal linguaggio

TdE 07/06/2023 – Quesito **a. - soluzione**

```
public class DataBuffer {
  private double[] data;
  private int cursor;

  public DataBuffer() { data = new double[100]; cursor = 0; }

  public double getDataAtCursor() {
    synchronized(data) { return data[cursor]; }
  }

  public void setDataAtCursor(double d) {
    synchronized(data) { data[cursor] = d; }
  }

  public void setCursorPos(int pos) {
    synchronized(data) { cursor = pos; }
  }
}
```

TdE 07/06/2023 – Quesito **a. - soluzione**

```
public class DataBuffer {
  private double[] data;
  private int cursor;

  public DataBuffer() { data = new double[100]; cursor = 0; }

  public double getDataAtCursor() {
    synchronized(data) { return data[cursor]; }
  }

  public void setDataAtCursor(double d) {
    synchronized(data) { data[cursor] = d; }
  }

  public void setCursorPos(int pos) {
    synchronized(data) { cursor = pos; }
  }
}
```

TdE 07/06/2023 – Quesito **b.**

Si modifichi il metodo getDataAtCursor in modo da sospendere il chiamante se il dato da ritornare risultasse minore di zero. Il chiamante verrà sbloccato quando il dato diventerà positivo.

TdE 07/06/2023 – Quesito **b. - soluzione**

Si modifichi il metodo getDataAtCursor in modo da sospendere il chiamante se il dato da ritornare risultasse minore di zero. Il chiamante verrà sbloccato quando il dato diventerà positivo.

```
public double getDataAtCursor1() throws InterruptedException {
  synchronized(data) {
    while(data[cursor]<0) data.wait();
    return data[cursor];
  }
}
```

TdE 07/06/2023 – Quesito **b. - soluzione**

Si modifichi il metodo getDataAtCursor in modo da sospendere il chiamante se il dato da ritornare risultasse minore di zero. Il chiamante verrà sbloccato quando il dato diventerà positivo.

```
public double getDataAtCursor1() throws InterruptedException {
   synchronized(data) {
     while(data[cursor]<0) data.wait();
     return data[cursor];
   }
}
```

Abbiamo messo una wait, quindi dobbiamo mettere anche
Una notifyAll() per poter svegliare il thread quando serve.
In questo caso, il thread deve essere svegliato quando
Il dato diventa positivo

TdE 07/06/2023 – Quesito **b. - soluzione**

Si modifichi il metodo getDataAtCursor in modo da sospendere il chiamante se il dato da ritornare risultasse minore di zero. Il chiamante verrà sbloccato quando il dato diventerà positivo.

```
public double getDataAtCursor1() throws InterruptedException {
  synchronized(data) {
    while(data[cursor]<0) data.wait();
    return data[cursor];
  }
}
public void setDataAtCursor(double d) {
   synchronized(data) {
     data[cursor] = d;
     if(d>=0) data.notifyAll();
   }
}
```

TdE 07/06/2023 – Quesito **c**

Si consideri il seguente metodo della classe DataBuffer che crea due thread e li esegue:

```
public void change() {
  new Thread(() -> {
    for(int i=0; i<100; i++) {
      setCursorPos(i);
      setDataAtCursor(i);
    }
  }).start();
  new Thread(() -> {
    for(int i=0; i<100; i++) {
      setCursorPos(i);
      setDataAtCursor(i*2);
    }
  }).start();
}
```

E’ corretto affermare che al termine dell’esecuzione dei due thread il contenuto dell’array data contiene in ogni posizione **i** il valore **i** oppure il valore **i*2**?
Motivare la propria risposta e in caso di risposta negativa modificare i thread affinchè la proprietà sia rispettata.

TdE 07/06/2023 – Quesito **c**

```
public void change() {
  new Thread(() -> {
    for(int i=0; i<100; i++) {
      synchronized(data) {
        setCursorPos(i);
        setDataAtCursor(i);
      }
    }
  }).start();

  new Thread(() -> {
    for(int i=0; i<100; i++) {
      synchronized(data) {
        setCursorPos(i);
        setDataAtCursor(i*2);
      }
    }
  }).start();
}
```

La proprietà non è rispettata perchè uno dei due thread potrebbe impostare il cursore a una certa posizione per poi, prima di impostare il dato nella posizione del cursore, cedere il controllo all'altro thread che potrebbe a sua volta impostare il dato con un valore che non sarà più "sincronizzato" con la posizione del cursore.

TdE 07/06/2023 – Quesito **c**

```
public void change() {
  new Thread(() -> {
    for(int i=0; i<100; i++) {
      synchronized(data) {
        setCursorPos(i);
        setDataAtCursor(i);
      }
    }
  }).start();

  new Thread(() -> {
    for(int i=0; i<100; i++) {
      synchronized(data) {
        setCursorPos(i);
        setDataAtCursor(i*2);
      }
    }
  }).start();
}
```

La proprietà non è rispettata perchè uno dei due thread potrebbe impostare il cursore a una certa posizione per poi, prima di impostare il dato nella posizione del cursore, cedere il controllo all'altro thread che potrebbe a sua volta impostare il dato con un valore che non sarà più "sincronizzato" con la posizione del cursore.

TdE 13/02/2023

Si consideri la seguente classe Products che contiene informazioni su un insieme di prodotti. Ogni prodotto `e caratterizzato da un identificativo e un nome univoci e un prezzo.

```
public class Products {
  private Map<Integer, String> idToName;
  private Map<String, Integer> nameToPrice;
  private int lastId;

  public Products() {
    idToName = new HashMap<>();
    nameToPrice = new HashMap<>();
    lastId = 0;
  }

  public int addProduct(String name, int price) {
    int assignedId = 0;
    synchronized (idToName) {
      assignedId = lastId++;
      idToName.put(assignedId, name);
    }
    synchronized (nameToPrice) { nameToPrice.put(name, price); }
    return assignedId;
  }

  // Ritorna il prezzo del prodotto avente l'id passato per parametro
  // Se tale prodotto non esiste, ritorna -1
  public int getPriceById(int id) {
    synchronized (idToName) {
      if (idToName.containsKey(id)) {
        String name  = idToName.get(id);
        return nameToPrice.get(name);
      } else {
        return -1;
      }
    }
  }
```

Quesito a.

Si dica se la classe Products è opportunamente sincronizzata per permettere l'invocazione concorrente dei suoi metodi da parte di più thread.

In caso negativo, si modifichi la classe per correggere la sua sincronizzazione

```
public class Products {
  private Map<Integer, String> idToName;
  private Map<String, Integer> nameToPrice;
  private int lastId;

  public Products() {
    idToName = new HashMap<>();
    nameToPrice = new HashMap<>();
    lastId = 0;
  }

  public int addProduct(String name, int price) {
    int assignedId = 0;
    synchronized (idToName) {
      assignedId = lastId++;
      idToName.put(assignedId, name);
    }
    synchronized (nameToPrice) { nameToPrice.put(name, price); }
    return assignedId;
  }

  // Ritorna il prezzo del prodotto avente l'id passato per parametro
  // Se tale prodotto non esiste, ritorna -1
  public int getPriceById(int id) {
    synchronized (idToName) {
      if (idToName.containsKey(id)) {
        String name = idToName.get(id);
        return nameToPrice.get(name);
      } else {
        return -1;
      }
    }
  }
}
```

Quesito a.

Si dica se la classe Products è opportunamente sincronizzata per permettere l'invocazione concorrente dei suoi metodi da parte di più thread.

In caso negativo, si modifichi la classe per correggere la sua sincronizzazione

La sincronizzazione è sbagliata poiché il metodo getPriceById accede alla variabile nameToPrice prendendo il lock solo su idToName.

Come posso risolvere?

TdE 13/02/2023 - Quesito **a. - soluzione**

```
public class Products {
  private Map<Integer, String> idToName;
  private Map<String, Integer> nameToPrice;
  private int lastId;

  public Products() {
    idToName = new HashMap<>();
    nameToPrice = new HashMap<>();
    lastId = 0;
  }

  public synchronized void addProduct(String name, int price) {
    int assignedId = lastId++;
    idToName.put(assignedId, name);
    nameToPrice.put(name, price);
    return assignedId;

  // Ritorna il prezzo del prodotto avente l'id passato per parametro
  // Se tale prodotto non esiste, ritorna -1
  public synchronized int getPriceById(int id) throws {
    if (idToName.containsKey(id)) {
      String name  = idToName.get(id);
      return nameToPrice.get(name);
    } else {
      return -1;
    }
```

La soluzione è rendere atomici i metodi utilizzando la keyword **synchronized**

TdE 13/02/2023 - Quesito **b**

```
public class Products {
  private Map<Integer, String> idToName;
  private Map<String, Integer> nameToPrice;
  private int lastId;

  public Products() {
    idToName = new HashMap<>();
    nameToPrice = new HashMap<>();
    lastId = 0;
  }

  public synchronized void addProduct(String name, int price) {
    int assignedId = lastId++;
    idToName.put(assignedId, name);
    nameToPrice.put(name, price);
    return assignedId;
  }
   // Ritorna il prezzo del prodotto avente l'id passato per parametro
   // Se tale prodotto non esiste, ritorna -1
   public synchronized int getPriceById(int id) throws {
     if (idToName.containsKey(id)) {
       String name  = idToName.get(id);
       return nameToPrice.get(name);
     } else {
       return -1;
     }
```

A partire dalla soluzione data alla domanda (a), si modifichi il metodo getPriceById in modo che, invece di ritornare -1, sospenda il chiamante se il prodotto non esiste. Se necessario, si indichi come modificare altri metodi.

TdE 13/02/2023 - Quesito **b - soluzione**

```
public synchronized int getPriceById(int id) throws InterruptedException {
  while (!idToName.containsKey(id)) {
    wait(); // Sospende il chiamante se l'id non e' presente
  }
  String name  = idToName.get(id);
  return nameToPrice.get(name);
}
```

TdE 13/02/2023 - Quesito **b - soluzione**

```
public synchronized int getPriceById(int id) throws InterruptedException {
  while (!idToName.containsKey(id)) {
    wait(); // Sospende il chiamante se l'id non e' presente
  }
  String name  = idToName.get(id);
  return nameToPrice.get(name);
}
```

Abbiamo aggiunto una wait. Quindi dovremo anche mettere una **notifyAll**()

TdE 13/02/2023 - Quesito **b - soluzione**

```
public synchronized int getPriceById(int id) throws InterruptedException {
  while (!idToName.containsKey(id)) {
    wait(); // Sospende il chiamante se l'id non e' presente
  }
  String name  = idToName.get(id);
  return nameToPrice.get(name);
}

public synchronized void addProduct(String name, int price) {
  int assignedId = lastId++;
  idToName.put(assignedId, name);
  nameToPrice.put(name, price);
  notifyAll(); // Risveglia potenziali thread in attesa di questo id
  return assignedId;
}
```

TdE 13/02/2023 - Quesito **c**

Si scriva un metodo **void copyPrices(Products other)** che, per tutti i prodotti di *this* che sono anche contenuti in other sostituisce i prezzi contenuti in this con i prezzi contenuti in other.
Si presti particolare attenzione a evitare il rischio di possibili deadlock.

Si ricordano alcuni metodi di Map che
- Map ha un costruttore che prende in ingresso un'altra mappa e ne copia il contenuto (chiavi e valori);
- Il metodo keySet() ritorna un Set con l'insieme delle chiavi;
- Il metodo put prende in ingresso una chiave e un valore e aggiunge o sovrascrive il valore associato alla chiave.

TdE 13/02/2023 - Quesito **c - soluzione**

```
void copyPrices(Products other) {
   Map<String, Integer> copy = null;
   synchronized (other) {
      copy = new HashMap<>(other.nameToPrice);
   }
   synchronized (this) {
      for (String name : copy.keySet()) {
         if (nameToPrice.containsKey(name)) {
            nameToPrice.put(name, copy.get(name));
         }
      }
   }
}
```

TdE 20/07/2018

Esercizio 2

Si consideri la seguente classe Java che realizza il tipo `Coppia`.

```
   public class Coppia {
1.    private double elem1, elem2;
2.    private Object lock;

3.    public Coppia() { elem1 = elem2 = 0; }

4.    public synchronized double get1() {
5.        return elem1;
6.    }

7.    public synchronized void set1(double val) {
8.        elem1 = val;
9.    }

10.   public void set2(double val) {
11.       synchronized(lock) {
12.           elem2 = val;
13.           notifyAll();
14.       }
15.   }

16.   public void waitForNegative() throws InterruptedException {
17.       synchronized(lock) {
18.           while(elem2>=0) wait();
19.       }
```

TdE 20/07/2018 - quesito a

- Si indichi se le seguenti affermazioni sono vere o false, motivando brevemente la propria risposta:

TdE 20/07/2018 - quesito a: Soluzione

- Si indichi se le seguenti affermazioni sono vere o false, motivando brevemente la propria risposta:

1. I metodi get1 e set1 non posso eseguire in parallelo.

TdE 20/07/2018

Esercizio 2

Si consideri la seguente classe Java che realizza il tipo `Coppia`.

```
   public class Coppia {
1.    private double elem1, elem2;
2.    private Object lock;

3.    public Coppia() { elem1 = elem2 = 0; }

4.    public synchronized double get1() {
5.        return elem1;
6.    }

7.    public synchronized void set1(double val) {
8.        elem1 = val;
9.    }

10.   public void set2(double val) {
11.       synchronized(lock) {
12.           elem2 = val;
13.           notifyAll();
14.       }
15.   }

16.   public void waitForNegative() throws InterruptedException {
17.       synchronized(lock) {
18.           while(elem2>=0) wait();
19.       }
```

TdE 20/07/2018 - quesito a: Soluzione

- Si indichi se le seguenti affermazioni sono vere o false, motivando brevemente la propria risposta:

1. I metodi get1 e set1 non posso eseguire in parallelo.

Vero, i due metodi sono sincronizzati sullo stesso oggetto, this.

TdE 20/07/2018 - quesito a: Soluzione

- Si indichi se le seguenti affermazioni sono vere o false, motivando brevemente la propria risposta:

2. I metodi set2 e waitForNegative non possono eseguire in parallelo

TdE 20/07/2018

Esercizio 2

Si consideri la seguente classe Java che realizza il tipo `Coppia`.

```
   public class Coppia {
1.    private double elem1, elem2;
2.    private Object lock;

3.    public Coppia() { elem1 = elem2 = 0; }

4.    public synchronized double get1() {
5.        return elem1;
6.    }

7.    public synchronized void set1(double val) {
8.        elem1 = val;
9.    }

10.   public void set2(double val) {
11.       synchronized(lock) {
12.           elem2 = val;
13.           notifyAll();
14.       }
15.   }

16.   public void waitForNegative() throws InterruptedException {
17.       synchronized(lock) {
18.           while(elem2>=0) wait();
19.       }
```

TdE 20/07/2018 - quesito a: Soluzione

- Si indichi se le seguenti affermazioni sono vere o false, motivando brevemente la propria risposta:

2. I metodi set2 e waitForNegative non possono eseguire in parallelo

Vero, i due metodi sono sincronizzati sullo stesso oggetto (this.lock)

TdE 20/07/2018 - quesito a: Soluzione

- Si indichi se le seguenti affermazioni sono vere o false, motivando brevemente la propria risposta:

3. I metodi set1 e set2 on possono eseguire in parallelo

TdE 20/07/2018

Esercizio 2

Si consideri la seguente classe Java che realizza il tipo `Coppia`.

```
   public class Coppia {
1.    private double elem1, elem2;
2.    private Object lock;

3.    public Coppia() { elem1 = elem2 = 0; }

4.    public synchronized double get1() {
5.        return elem1;
6.    }

7.    public synchronized void set1(double val) {
8.        elem1 = val;
9.    }

10.   public void set2(double val) {
11.       synchronized(lock) {
12.           elem2 = val;
13.           notifyAll();
14.       }
15.   }

16.   public void waitForNegative() throws InterruptedException {
17.       synchronized(lock) {
18.           while(elem2>=0) wait();
19.       }
```

TdE 20/07/2018 - quesito a: Soluzione

- Si indichi se le seguenti affermazioni sono vere o false, motivando brevemente la propria risposta:

3. I metodi set1 e set2 non possono eseguire in parallelo

Falso, i due metodi sono sincronizzati su oggetti diversi

TdE 20/07/2018 - quesito a: Soluzione

- Si indichi se le seguenti affermazioni sono vere o false, motivando brevemente la propria risposta:

4. Il codice compila ed esegue senza errori. In caso negativo evidenziare. sfruttando i numeri di linea indicati nel testo, quali linee vadano modificate (e come) per evitare gli errori.

TdE 20/07/2018

Esercizio 2

Si consideri la seguente classe Java che realizza il tipo `Coppia`.

```
   public class Coppia {
1.    private double elem1, elem2;
2.    private Object lock;

3.    public Coppia() { elem1 = elem2 = 0; }

4.    public synchronized double get1() {
5.        return elem1;
6.    }

7.    public synchronized void set1(double val) {
8.        elem1 = val;
9.    }

10.   public void set2(double val) {
11.       synchronized(lock) {
12.           elem2 = val;
13.           notifyAll();
14.       }
15.   }

16.   public void waitForNegative() throws InterruptedException {
17.       synchronized(lock) {
18.           while(elem2>=0) wait();
19.       }
```

TdE 20/07/2018 - quesito a: Soluzione

4. Il codice compila ed esegue senza errori. In caso negativo evidenziare. sfruttando i numeri di linea indicati nel testo, quali linee vadano modificate (e come) per evitare gli errori.

Il codice compila correttamente ma ha un errore **run-time** eseguendo i metodi **set2** e **waitForNegative**.
L'errore deriva dall'invocare wait e notifyAll su this quando si ha il lock sull'oggetto this.lock ma non su this. La soluzione prevede di sostituire la linea 13 come segue:

13. lock.notifyAll()

Analogamente la linea 18 diventa:

18. while(elem2>=0) lock.wait();

TdE 20/07/2018 - quesito b

- Si aggiunga, alla classe sviluppata al punto precedente, un metodo public void set12() che imposta il valore dell'attributo elem2 al doppio del valore dell'attributo elem1. Nel fare ciò, se necessario, si modifichi il codice di sincronizzazione in maniera da evitare conflitti nell'accesso agli attributi condivisi da parte dei vari metodi (incluso il nuovo).

TdE 20/07/2018 - quesito b: Soluzione

- Devo creare quindi un metodo set12 che usi sia set1 che set2. Attualmente però, set1 sfrutta la sincronizzazione su this, set2 sfrutta la sincronizzazione su lock…

  come va sincronizzato correttamente il metodo? Devo cambiare altro?

TdE 20/07/2018 - quesito b: Soluzione

Soluzione

Il nuovo metodo `set12` deve accedere a entrambi gli attributi `elem1` ed `elem2`, occorre quindi sincronizzarlo rispetto a tutti gli altri metodi. Ciò può essere ottenuto cambiando il codice di sincronizzazione in maniera da sincronizzare tutti i metodi su `this` come segue:

```
public class Coppia {
    ....
    public synchronized double get1() {
        return elem1;
    }

    public synchronized void set1(double val) {
        elem1 = val;
    }

    public synchronized void set2(double val) {
        elem2 = val;
        notifyAll();
    }

    public synchronized void waitForNegative() throws InterruptedException {
        while(elem2>=0) wait();
    }

    public synchronized void set12() {
        elem2 = 2*elem1;
        notifyAll();
    }
}
```

TdE 18/06/22

Si consideri la seguente versione semplificata delle classi `HotelDB` e `Hotel`.

```
class HotelDB {
  private final List<Hotel> hotels;

  public HotelDB(List<Hotel> hotels) {
    this.hotels = new ArrayList<>(hotels);
  }

  public void book(int id) {
    if (hotels.get(id).getAvailableRooms() > 0) {
      hotels.get(id).bookOneRoom();
  } }
}

class Hotel {
  private int totalRooms;
  private int availableRooms;

  public synchronized int getAvailableRooms() { return availableRooms; }

  public synchronized int getTotalRooms() { return totalRooms; }

  public synchronized void bookOneRoom() { availableRooms--; }

  public synchronized void cancelOneBooking() { availableRooms++; }
}
```

TdE 18/06/22 - quesito a

L’implementazione fornita risulta correttamente sincronizzata? In caso contrario, indicare come correggere l’implementazione, favorendo soluzioni che massimizzino l’esecuzione parallela di thread che accedono ai metodi della classe HotelDB.

TdE 18/06/22 - quesito a

L’implementazione fornita risulta correttamente sincronizzata? In caso contrario, indicare come correggere l’implementazione, favorendo soluzioni che massimizzino l’esecuzione parallela di thread che accedono ai metodi della classe HotelDB.

```
public void book(int id) {
    if (hotels.get(id).getAvailableRooms() > 0) {
        hotels.get(id).bookOneRoom();
} }
```

Questo è **di fatto** un hotel, ma NON STO SINCRONIZZANDO

TdE 18/06/22 - quesito a: Soluzione

Soluzione

L'implementazione *non* risulta correttamente sincronizzata, in quanto il controllo della disponibilità di stanze e la successiva prenotazione non avvengono in maniera atomica. Di conseguenza il numero di stanze disponibili in un hotel potrebbe andare sotto 0. Per correggere, si può modificare l'implementazione come mostrato di seguito.

```
class HotelDB {
  ...
  public void book(int id) {
    Hotel h = hotels.get(id);
    synchronized(h) {
      if (h.getAvailableRooms() > 0) {
        h.bookOneRoom();
  } } }
}
```

TdE 16/07/2019

Si consideri la seguente classe Numbers che implementa una sequenza di numeri reali di dimensioni fissata al momento della costruzione. Il metodo average calcola la media dei due elementi in posizione pos1 e pos2 impostando a tale valor medio entrambi gli elementi.

```
public class Numbers {
  private double data[];
  private Object lock[];

  public Numbers(int size) {
    data = new double[size];
    lock = new Object[size];
    for (int i=0; i<size; i++) lock[i] = new Object();
  }

  public void set(int pos, double val) {
    synchronized(lock[pos]) { data[pos] = val; }
  }

  public double get(int pos) {
    synchronized(lock[pos]) { return data[pos]; }
  }

  public void average(int pos1, int pos2) {
    synchronized(lock[pos1]) {
      synchronized(lock[pos2]) {
        double avg = (data[pos1] + data[pos2]) / 2.0;
        data[pos1] = data[pos2] = avg;
      }
    }
  }
}
```

TdE 16/07/2019 - quesito a: soluzione

La classe Numbers presenta un problema di sincronizzazione potendo dar luogo a deadlock. Si scriva una sequenza di chiamate da parte di due thread paralleli che può portare a un deadlock e si indichi come modificare il codice della classe Numbers per evitare il rischio di deadlock pur mantenendo una sincronizzazione che eviti conflitti nell'accesso ai dati.

TdE 16/07/2019 - quesito a: soluzione

Il deadlock deriva dalla doppia sincronizzazione del metodo average. `E sufficiente che un thread invochi average(i,j) e l'altro average(j,i) per i medesimi valori di i e j perchè il deadlock possa manifestarsi.

Una possibile soluzione consiste nell'eliminare la sincronizzazione con oggetti espliciti (attributo lock) sincronizzando tutti i metodi su this.

TdE 16/07/2019 - quesito a: soluzione

```
public class NumbersOk {
  private double data[];

  public NumbersOk(int size) {
    data = new double[size];
  }

  public synchronized void set(int pos, double val) {
    data[pos] = val;
    notifyAll();
  }

  public synchronized double get(int pos) {
    return data[pos];
  }

  public synchronized void average(int pos1, int pos2) {
    double avg = (data[pos1] + data[pos2]) / 2.0;
    data[pos1] = data[pos2] = avg;
  }
```

TdE 16/07/2019 - quesito b

- Sulla base della versione corretta dela classe Numbers elaborata al punto precedente, si aggiunga un metodo waitForPositiveSum() che sospende il chiamante fino a quando la somma di tutti gli elementi in Numbers è negativa. Si devono modificare altri metodi della classe Numbers?

TdE 16/07/2019 - quesito b - Soluzione

```
public synchronized void waitForPositiveSum() throws InterruptedException {
  double sum;
  do {
    sum = 0.0;
    for (int i=0; i<data.length; i++) sum += data[i];
    if (sum<0.0) wait();
  } while (sum>0.0);
}
```